

CATÁLOGO DE TIENDA AVANZADO

EN JAVA

AGREGACIÓN DEL ENCAPSULAMIENTO

Este documento explica únicamente los cambios realizados al proyecto "CatalogoTiendaAvanzado" para agregar el concepto de encapsulamiento, manteniendo la estructura y funcionamiento original del programa.

1. Encapsulamiento

Se agregó el encapsulamiento en la clase abstracta ItemTienda. Anteriormente, los atributos nombre y costo estaban declarados como protected. Para proteger mejor la información de la clase, se cambiaron a private. De esta manera, estos atributos no pueden ser modificados directamente desde otras clases.

2. Atributos privados

Los atributos principales de ItemTienda ahora quedan de la siguiente forma:

```
private String nombre;  
private int costo;
```

El modificador private permite proteger los datos y controlar el acceso a ellos mediante métodos específicos.

3. Constructor y referencia this

El constructor que ya tenía el proyecto se mantiene. Su función es recibir el nombre y el costo del producto e inicializar los atributos. También se mantiene la referencia this, que indica que se está haciendo referencia al atributo del objeto actual.

```
public ItemTienda(String nombre, int costo) {  
    this.nombre = nombre;  
    this.costo = costo;  
}
```

4. Getters

Se mantienen los métodos getters que ya estaban presentes en el proyecto. Estos métodos permiten obtener el valor de los atributos privados sin acceder directamente a ellos.

```
public String getNombre() {  
    return nombre;  
}
```

```
public int getCosto() {  
    return costo;  
}
```

5. Setters

Se agregaron los métodos setters. Su función es permitir modificar los atributos privados de una forma controlada.

```
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}
```

```
public void setCosto(int costo) {  
    this.costo = costo;  
}
```

6. Método toString()

También se agregó el método toString(). Este permite representar la información del objeto como texto, mostrando principalmente su nombre y costo.

```
@Override  
public String toString() {  
    return "ItemTienda{" +  
        "nombre=" + nombre + "\" +  
        ", costo=" + costo +  
        "}";  
}
```

7. Clase ItemTienda modificada

Con estos cambios, la clase ItemTienda queda preparada para demostrar encapsulamiento, constructor, referencia this, getters, setters y toString(), sin eliminar el método abstracto utilizado anteriormente para el polimorfismo.

```
abstract class ItemTienda {  
  
    private String nombre;  
    private int costo;  
  
    public ItemTienda(String nombre, int costo) {  
        this.nombre = nombre;  
        this.costo = costo;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getCosto() {  
        return costo;  
    }  
  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void setCosto(int costo) {  
        this.costo = costo;  
    }  
}
```

```
@Override
public String toString() {
    return "ItemTienda{" +
        "nombre=" + nombre + "\" +
        ", costo=" + costo +
        "}";
}

public abstract void aplicarEfectoEspecial();
}
```

8. Relación con el proyecto original

Los cambios realizados no modifican la finalidad del catálogo ni la demostración de polimorfismo. Las clases `ItemAvatar` e `ItemPase` continúan heredando de `ItemTienda` y utilizando sus constructores. La diferencia principal es que ahora los atributos `nombre` y `costo` están protegidos mediante `private` y se accede a ellos mediante `getters` y `setters`.

9. Resumen del cambio

El proyecto original ya utilizaba constructores, la referencia `this` y métodos `getters`. Para completar la aplicación del encapsulamiento se realizaron tres cambios principales: los atributos `nombre` y `costo` pasaron de `protected` a `private`, se agregaron los `setters` para modificar sus valores y se incorporó el método `toString()` para mostrar la información del objeto.

De esta manera, el proyecto conserva su funcionamiento anterior y agrega los elementos necesarios para demostrar el encapsulamiento en Java.